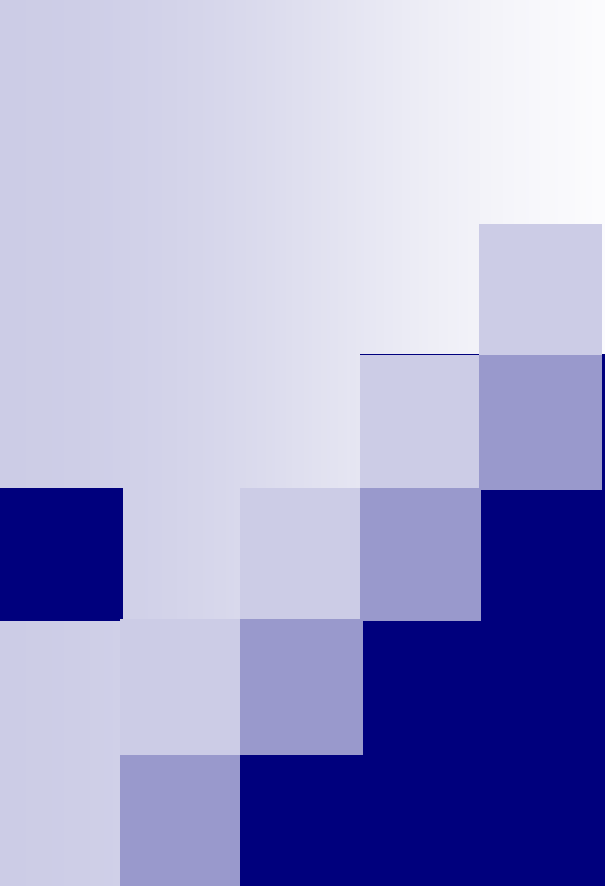




MODELLING CONCEPTS

Definitions

- **OOAD:-** is a software engineering approach that models a system as a group of interacting objects.
- It is used to model software systems as collections of co-operating objects, treating individual objects as instances of class within class hierarchy.

Definitions contd...

- OOA:- is a method of analysis that examines the requirements of a system from the classes and objects found in vocabulary of problem domain.
- OOA emphasizes building of real-world models using an object-oriented view.
- Also, OOA applies object modeling techniques to analyze function requirements of a system.
- OOA focuses on **WHAT** system does.

Definitions contd...

- **OOD**:- is a method of design encompassing the process of object-oriented decomposition and a notation for describing both logical & physical as well as static & dynamic models of system under design.
- OOD elaborates the analysis models to produce implementation specifications.
- OOD focuses on **HOW** system does it.
- **OML**:- is a standardized set of symbols and ways of arranging them to model an object oriented S/W design.



MODELING

- Model:- is pattern, plan ,representation , or description designed to show main object or workings of an object, system or concept.

IMPORTANCE

- Modeling is a proven and well-accepted engineering technique.

For e.g. we build architectural models for building houses and high- rises...

And mathematical models for analyzing effects of winds or earthquakes on buildings.

Modeling is also used in manufacturing automobiles and electrical devices from microprocessors to telephone switching systems.

- ***It means modeling is nothing but a simplification of reality.***



Contd..

- A model provides blue-prints of a system. i.e. it provides a **detailed plan** of action.
- A model may be structural or behavioral.
- The basic reason to model is:- **to understand the system we are developing in better way.**



OBJECTIVES

- Helps us to visualize a system as it is or as we want it to be.
- Permits us to specify the structure or behavior of a system.
- Gives us a template that guides us in constructing a system.
- It documents the decisions that we have made.



PRINCIPLES

- The choice of what models to create has a profound influence on how problem is attacked and how a solution is shaped.
- Every model may be expressed at different levels of precision.
- The best models are connected to reality.
- No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.



Similarly, to understand the architecture of a system, we need several complementary and interlocking views:-

Use Case view

Design View

Process View

Implementation View

Deployment View

Together these views represent the blueprints of a software.



Object oriented methodology

- It is a set of methods, models, and rules for developing systems
- Many methodologies are available to choose from
- Difference lies in documentation of information & modeling notations and languages.



OBJECT ORIENTED METHODOLOGIES

- **Rumbaugh's Object Modeling Technique** (well suited for describing the object model or the static structure of the system).
- **The Booch Methodology** (produces detailed object oriented design models).
- **The Jacobson Methodology** (well suited for producing use-case driven analysis models).

All the three methodologies are the origins of a UML.



UML FUNDAMENTALS

Unified Modelling Language

- Unified because it ...– Combines main preceding OO methods (Booch by Grady Booch, OMT by Jim Rumbaugh and OOSE by Ivar Jacobson)
- Modelling because it is ...– Primarily used for visually modelling systems. Many system views are supported by appropriate models
- Language because ...– It offers a syntax through which you express modelled knowledge



What is UML?

- A language for capturing and expressing knowledge
- A tool for system discovery and development
- A tool for visual development modelling
- A set of well-founded guidelines
- A milestone generator
- A popular (therefore supported) tool



UML is a language for

- Visualizing
 - Specifying
 - Constructing and
 - Documenting
- the artifacts of a software-intensive systems.



What UML is not!

- A visual programming language or environment
- A database specification tool
- A development process (i.e. an SDLC)



What UML can do for you

- Helps you to:—
- Better think out and document your system before implementing it
- “forecast” your system
- Determine islands of reusability
- Lower development costs



Contd...

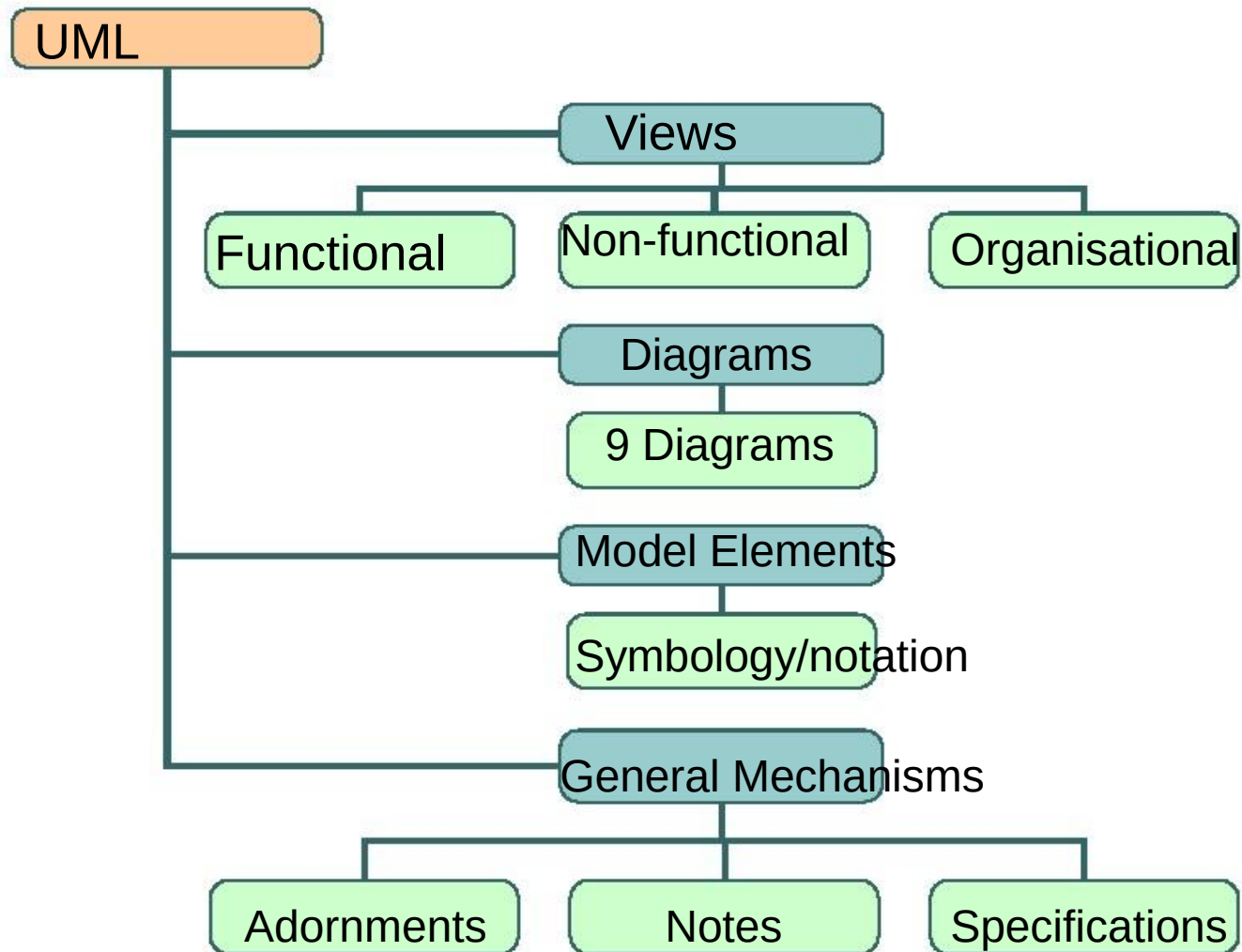
- Make the right decisions at an early stage (before committed to code)
- Better deploy the system for efficient memory and processor usage
- Plan and analyse your logic (system behaviour)



UML Partners

- Hewlett-Packard
- IBM
- Microsoft
- Oracle
- i-Logix
- Intelli Corp.
- MCI Systemhouse
- ObjectTime and many more

UML Components





The Case “for” Diagrams

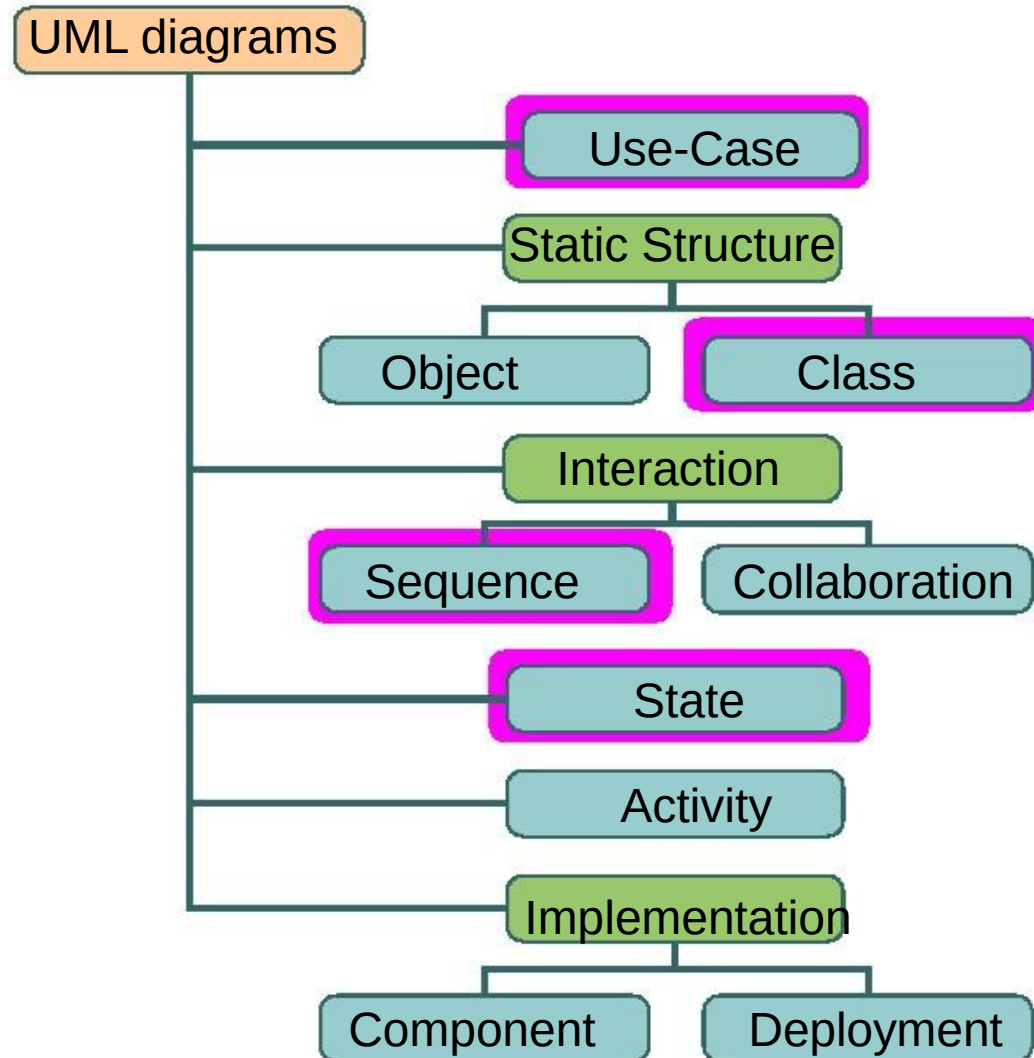
- Descriptive
- Compressive
- Simple
- Understandable
- Universal
- Formalise-able / Standardise-able



The Case “against” Diagrams

- Not inherent knowledge
- Easily cluttered
- Require some training
- Not necessarily revealing
- Must be liked to be accepted and used
- Effort to draw

UML DIAGRAMS CLASSIFICATION



UML Diagrams (comparative slide)

- Use-Case (relation of actors to system functions) [E.g.](#)
- Class (static class structure) [E.g.](#)
- Object (same as class - only using class instances— i.e. objects) [E.g.](#)
- State (states of objects in a particular class) [E.g.](#)
- Sequence (Object message passing structure) [E.g.](#)

Contd.....

- Collaboration (same as sequence but also shows context - i.e. objects and their relationships) E.g.
- Activity (sequential flow of activities i.e. action states) E.g.
- Component (code structure) E.g.
- Deployment (mapping of software to hardware) E.g.

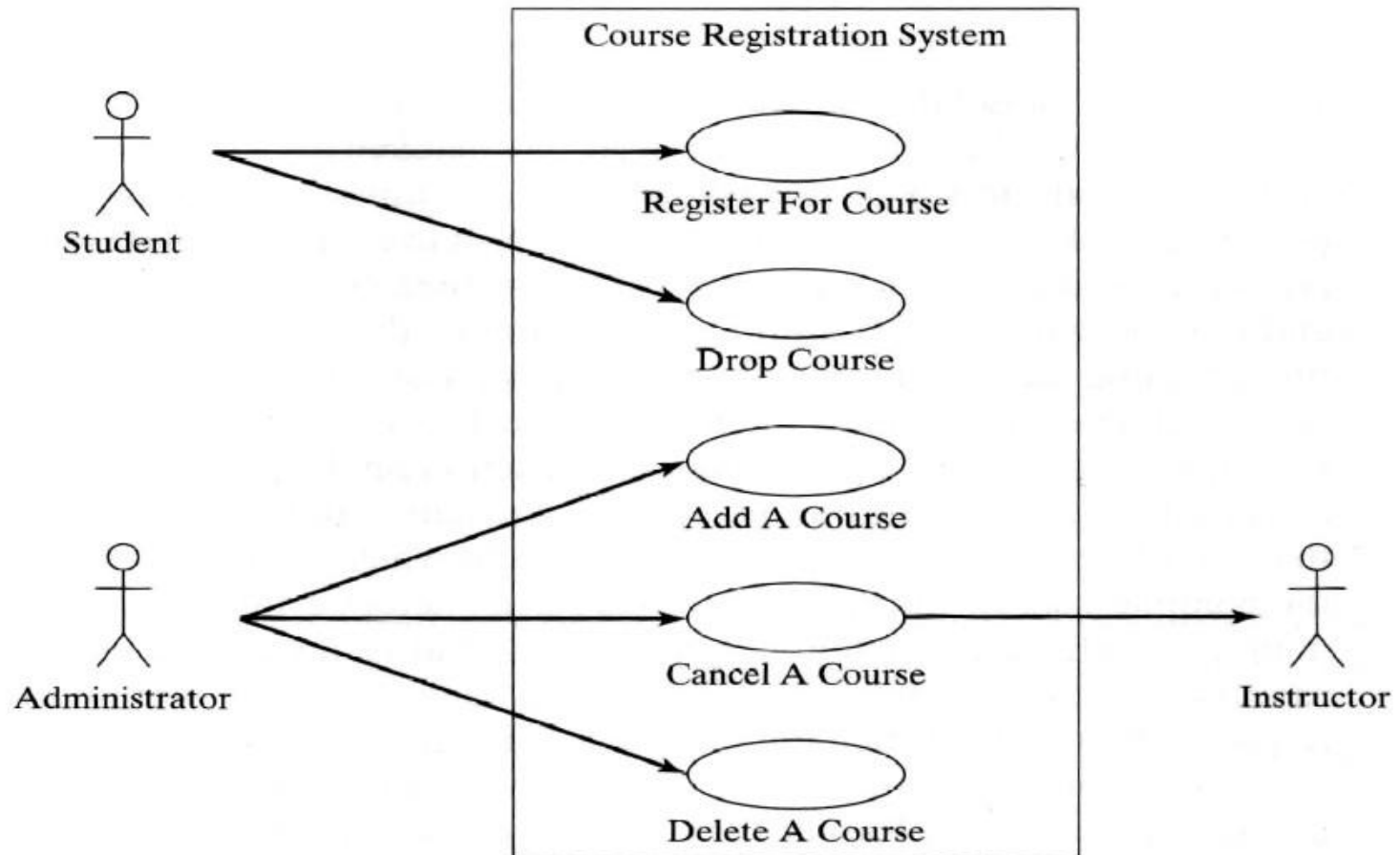


UML Diagram Philosophy

Any UML diagram:

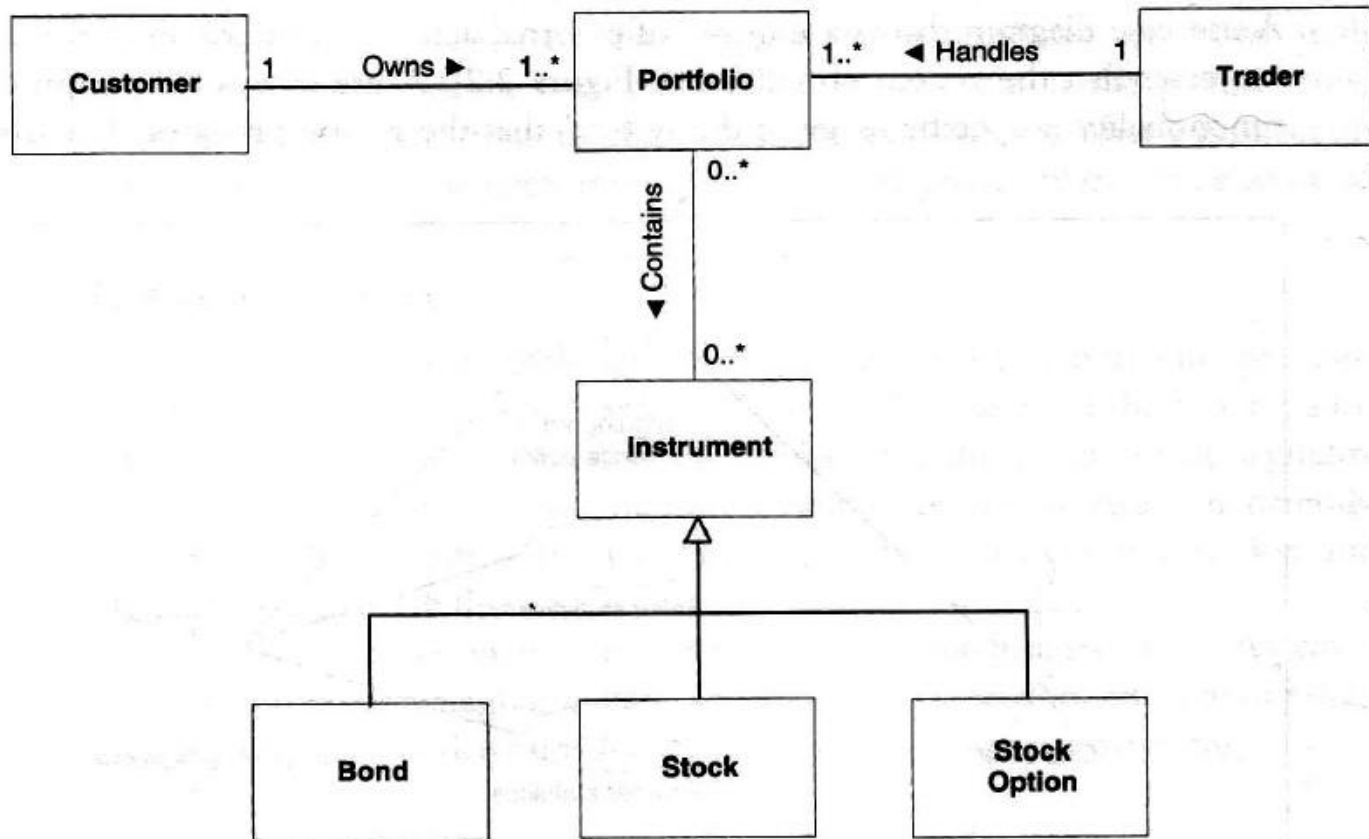
- Depicts concepts— as symbols
- Depicts relationships between concepts— as directed or undirected arcs (lines)
- Depicts names— as labels within or next to symbols and lines

Use Case Diagram

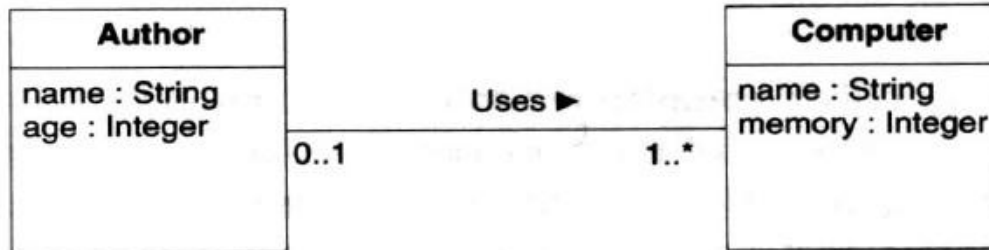


[Back](#)

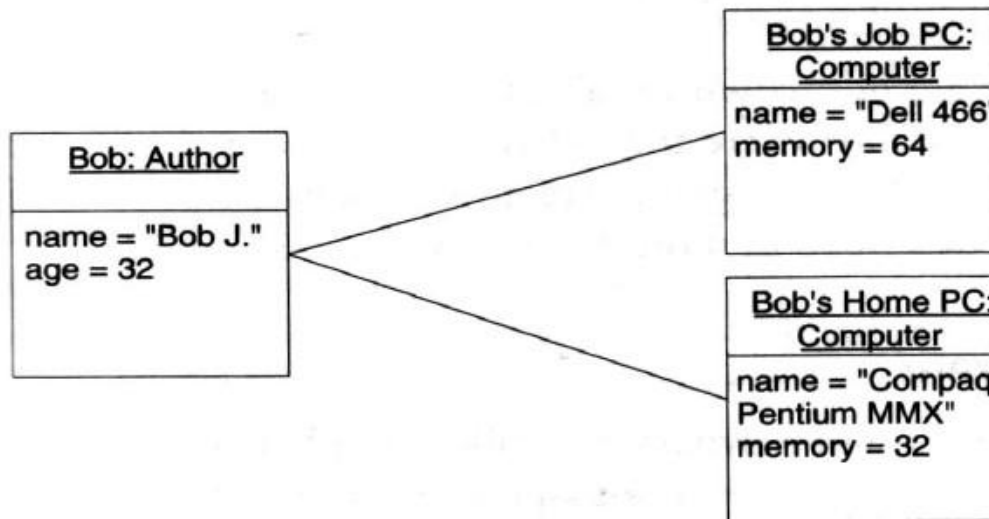
Class Diagram



Object Diagram

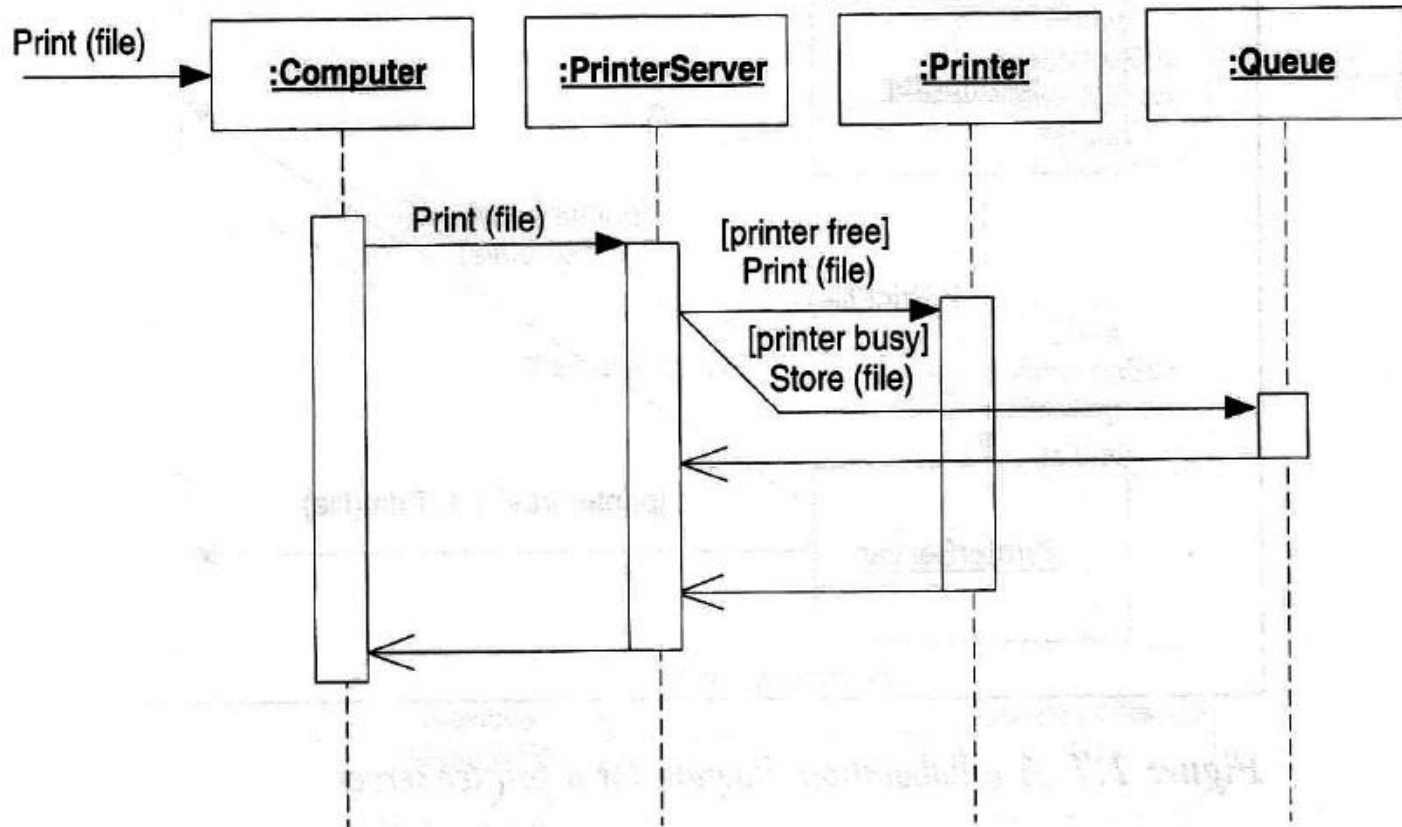


Class Diagram

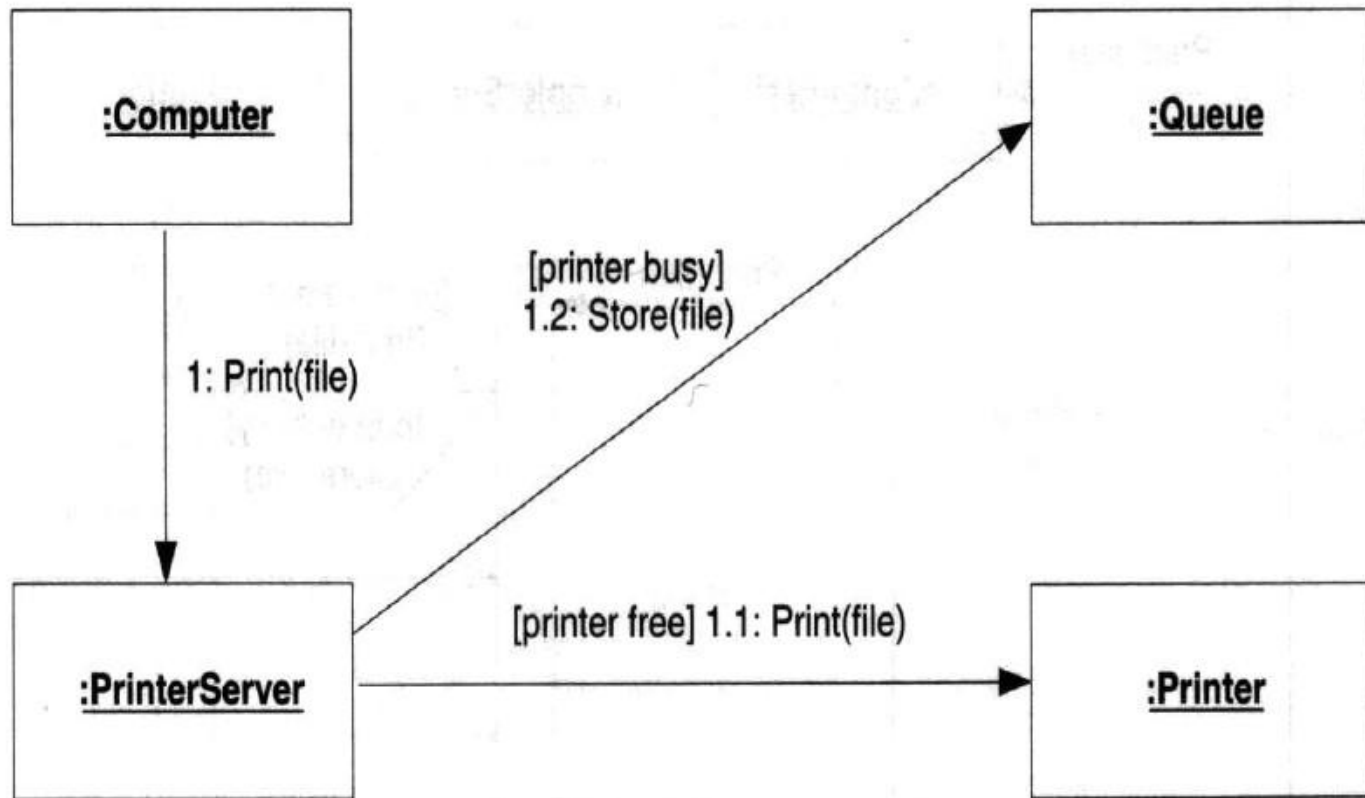


Object Diagram

Sequence Diagram

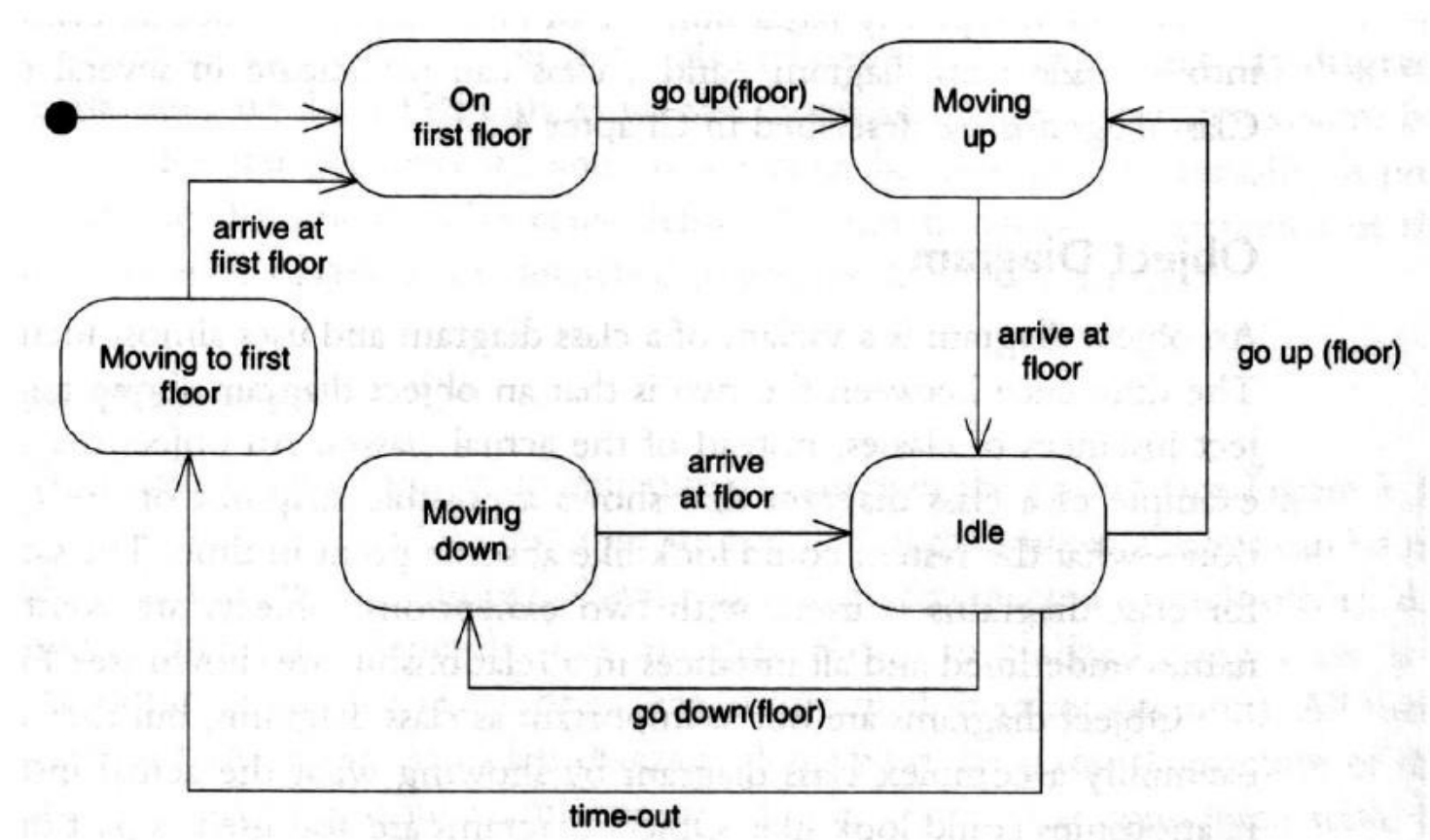


Collaboration Diagram



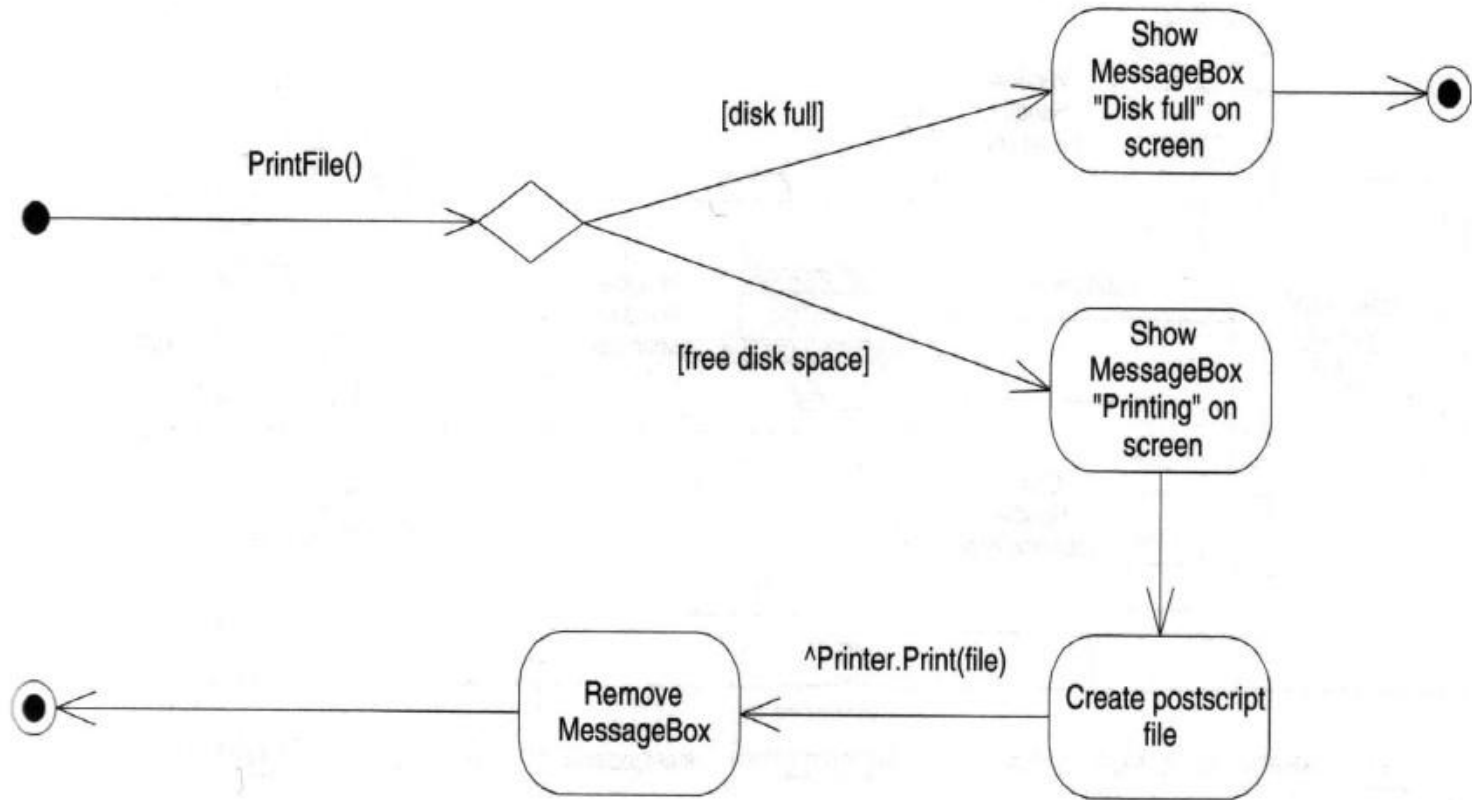
[Back](#)

State Diagram

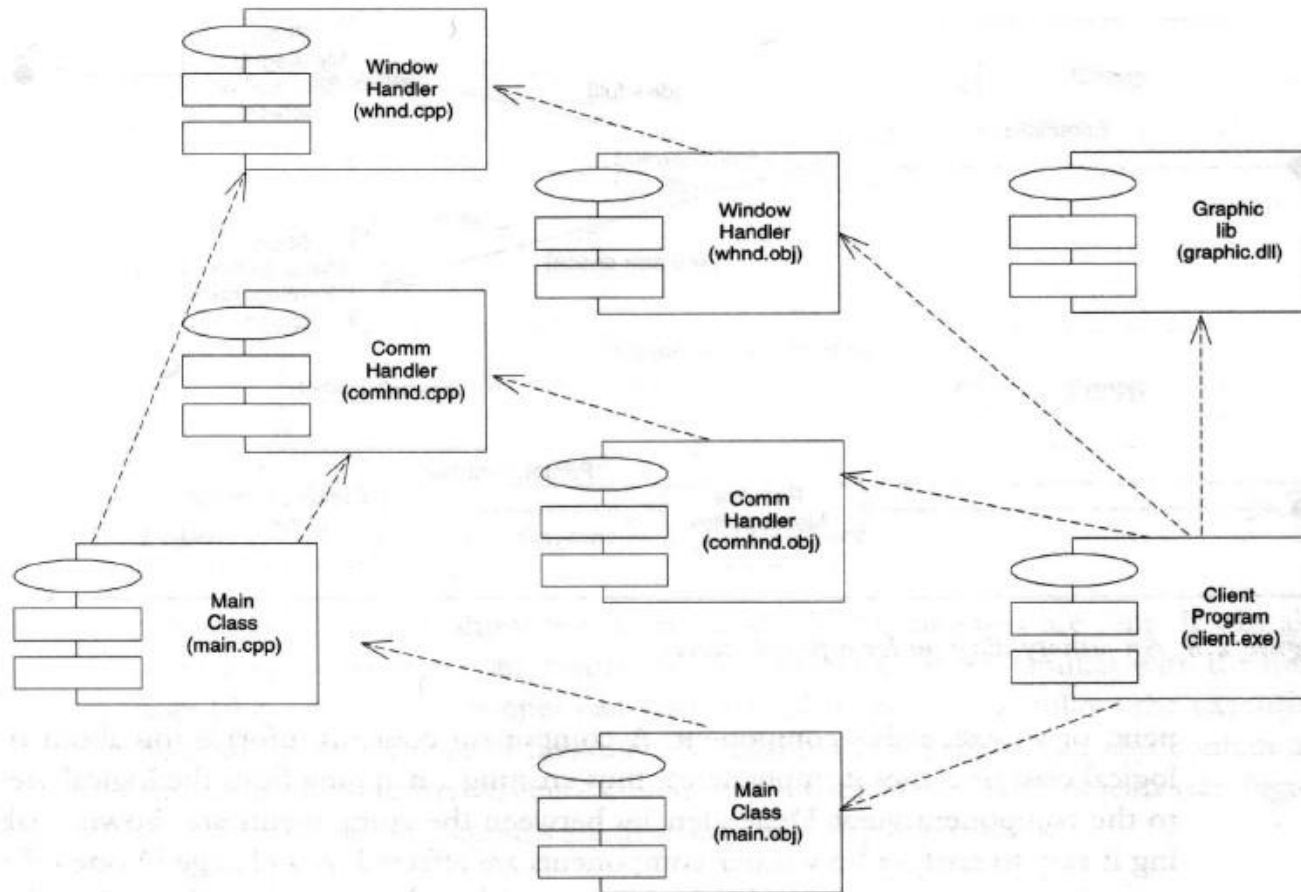


[Back](#)

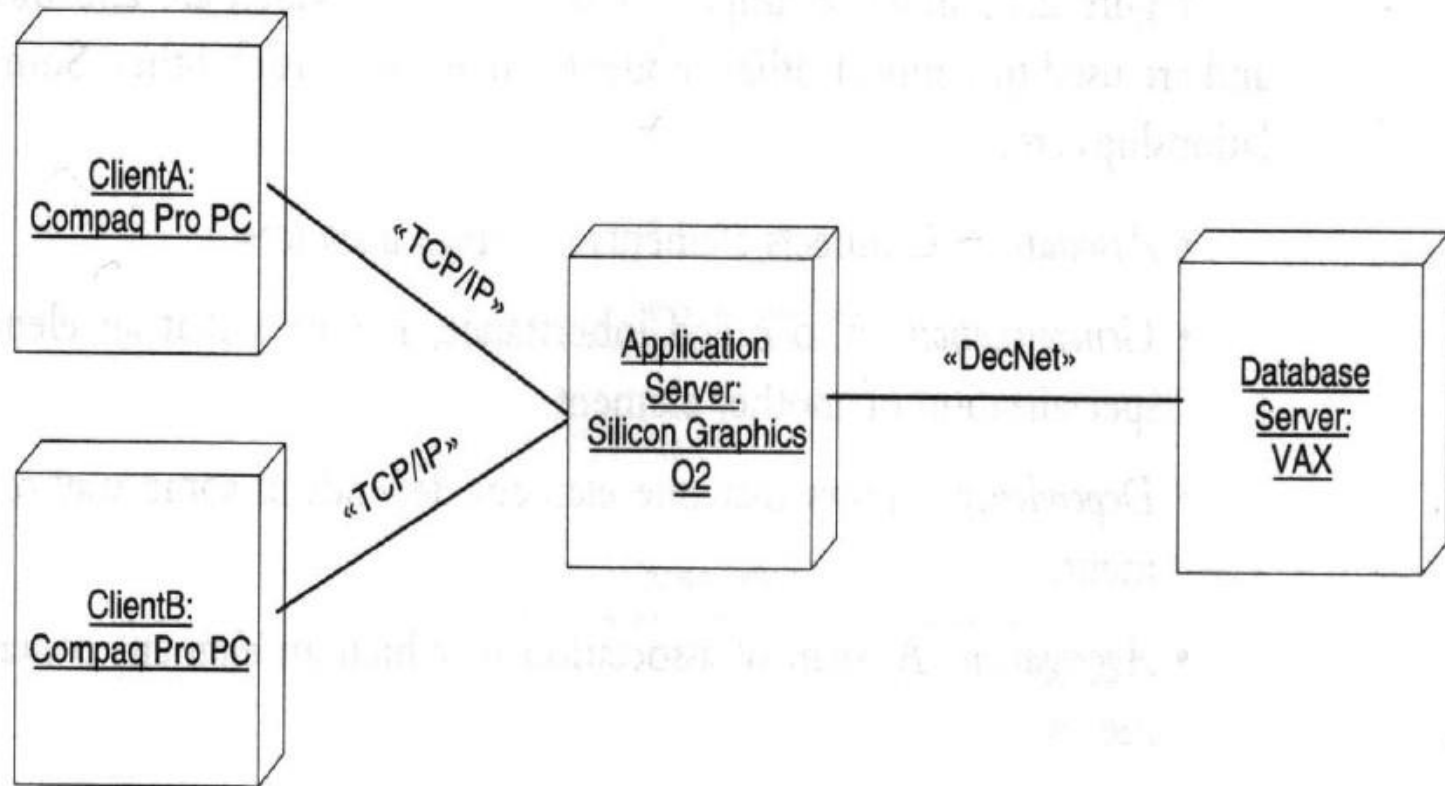
Activity Diagram



Component Diagram



Deployment Diagram



[Back](#)

Review Questions

- Define :
 - OOA
 - OOD
 - OOAD
 - OML
- Define Modeling . What are its objectives?
- What is UML? Explain role of different UML diagrams in system design? [May 2016]
- What is UML? List different UML diagrams.